

Q1)Implement a robust Java calculator program that captures user input dynamically, processes mathematical expressions using conditional logic and looping constructs, and ensures efficient error handling.

```
import java.util.Scanner;

public class Calculator {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        while (true) {

            try {

                System.out.println("\n--- Calculator ---");

                System.out.print("Enter first number: ");

                double num1 = sc.nextDouble();

                System.out.print("Enter operator (+, -, *, /, %, ^): ");

                char op = sc.next().charAt(0);

                System.out.print("Enter second number: ");

                double num2 = sc.nextDouble();

                double result;

                switch (op) {

                    case '+':

                        result = num1 + num2;

                        break;

                    case '-':

                        result = num1 - num2;

                        break;

                    case '*':

                        result = num1 * num2;

                        break;

                    case '/':

                        if (num2 == 0) {

                            System.out.println("Error: Division by zero!");

                            continue;

                        }

                        result = num1 / num2;

                }

            }

        }

    }

}
```

```

        break;
    case '%':
        result = num1 % num2;
        break;
    case '^':
        result = Math.pow(num1, num2);
        break;
    default:
        System.out.println("Invalid operator!");
        continue;
}
System.out.println("Result: " + result);

System.out.print("Do you want to continue? (y/n): ");
char choice = sc.next().charAt(0);
if (choice == 'n' || choice == 'N') {
    break;
}
} catch (Exception e) {
    System.out.println("Invalid input! Please try again.");
    sc.nextLine(); // clear buffer
}
}
sc.close();
System.out.println("Calculator Closed.");
}
}

```

Q2) Develop a Java program for an E-commerce order processing where some products are initialized through multiple constructors, users can input some product details manually, the system computes total order cost dynamically, applies discount policies based on conditions, and presents a detailed invoice summarizing the purchase.

```
import java.util.*;
```

```

class Product {
    int id;
    String name;
    double price;
    int quantity;
    // Default constructor
    Product() {
        this.id = 0;
        this.name = "Unknown";
        this.price = 0;
        this.quantity = 0;
    }
    // Parameterized constructor
    Product(int id, String name, double price, int quantity) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }
    double getTotal() {
        return price * quantity;
    }
}

public class Ecommerce {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Product> cart = new ArrayList<>();
        // Predefined products using constructor
        cart.add(new Product(1, "Laptop", 50000, 1));
        cart.add(new Product(2, "Mouse", 500, 2));
        System.out.print("Enter number of products to add: ");
    }
}

```

```

int n = sc.nextInt();
// User input products
for (int i = 0; i < n; i++) {
    System.out.println("\nEnter details for product " + (i + 1));
    System.out.print("ID: ");
    int id = sc.nextInt();
    sc.nextLine(); // clear buffer
    System.out.print("Name: ");
    String name = sc.nextLine();
    System.out.print("Price: ");
    double price = sc.nextDouble();
    System.out.print("Quantity: ");
    int qty = sc.nextInt();
    cart.add(new Product(id, name, price, qty));
}
double total = 0;
// Calculate total
for (Product p : cart) {
    total += p.getTotal();
}
double discount = 0;
// Discount logic
if (total > 100000) {
    discount = total * 0.15;
} else if (total > 50000) {
    discount = total * 0.10;
} else if (total > 20000) {
    discount = total * 0.05;
}
double finalAmount = total - discount;
// Invoice
System.out.println("\n===== INVOICE =====");

```

```

        System.out.println("ID\tName\tPrice\tQty\tTotal");
        for (Product p : cart) {
            System.out.println(p.id + "\t" + p.name + "\t" + p.price + "\t" + p.quantity + "\t" + p.getTotal());
        }
        System.out.println("-----");
        System.out.println("Total Amount: " + total);
        System.out.println("Discount: " + discount);
        System.out.println("Final Amount: " + finalAmount);
        sc.close();
    }
}

```

Q3) Write a Java program that demonstrates the overloading method to compute power and absolute of a number for various data types and utilizes the static method from Math class for the same operation

```

class OverloadDemo {
    // Power methods (overloading)
    static int power(int a, int b) {
        return (int) Math.pow(a, b);
    }
    static double power(double a, double b) {
        return Math.pow(a, b);
    }
    // Absolute methods (overloading)
    static int abs(int a) {
        return Math.abs(a);
    }
    static double abs(double a) {
        return Math.abs(a);
    }
    public static void main(String[] args) {
        // Using overloaded methods
        System.out.println("Power (int): " + power(2, 3));
        System.out.println("Power (double): " + power(2.5, 2));
    }
}

```

```

System.out.println("Absolute (int): " + abs(-10));
System.out.println("Absolute (double): " + abs(-5.7));
// Direct use of Math class
System.out.println("\nUsing Math class directly:");
System.out.println("Math.pow: " + Math.pow(3, 2));
System.out.println("Math.abs: " + Math.abs(-15));
}
}

```

Q4) Develop a Java program that performs various operations on arrays, including displaying elements, finding the maximum and minimum elements, calculating the sum and average of elements, and searching for a specific element within the array

```

import java.util.*;

public class ArrayOperations {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        // Input
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();
        // Display
        System.out.print("Array: ");
        for (int x : arr)
            System.out.print(x + " ");
        // Max, Min, Sum
        int max = arr[0], min = arr[0], sum = 0;
        for (int x : arr) {
            if (x > max) max = x;
            if (x < min) min = x;

```

```

        sum += x;
    }
    double avg = (double) sum / n;
    // Search
    System.out.print("\nEnter element to search: ");
    int key = sc.nextInt();
    boolean found = false;
    for (int x : arr) {
        if (x == key) {
            found = true;
            break;
        }
    }
    // Output
    System.out.println("Max: " + max);
    System.out.println("Min: " + min);
    System.out.println("Sum: " + sum);
    System.out.println("Average: " + avg);
    System.out.println("Element " + (found ? "found" : "not found"));
    sc.close();
}
}

```

Q5) Develop a Java program that implements a simple hotel room booking system using two-dimensional arrays. The system allows users to: View available and booked rooms, Book a room by selecting a floor and room number and exit the system when finished.

```

import java.util.*;

public class HotelBooking {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int floors = 3, rooms = 4;
        int[][] hotel = new int[floors][rooms]; // 0 = available, 1 = booked
        while (true) {

```

```

System.out.println("\n1.View Rooms 2.Book Room 3.Exit");
System.out.print("Enter choice: ");
int ch = sc.nextInt();
switch (ch) {
    case 1:
        System.out.println("\nRoom Status (0=Available, 1=Booked):");
        for (int i = 0; i < floors; i++) {
            System.out.print("Floor " + (i + 1) + ": ");
            for (int j = 0; j < rooms; j++)
                System.out.print(hotel[i][j] + " ");
            System.out.println();
        }
        break;
    case 2:
        System.out.print("Enter floor (1-" + floors + "): ");
        int f = sc.nextInt() - 1;
        System.out.print("Enter room (1-" + rooms + "): ");
        int r = sc.nextInt() - 1;
        if (f >= 0 && f < floors && r >= 0 && r < rooms) {
            if (hotel[f][r] == 0) {
                hotel[f][r] = 1;
                System.out.println("Room booked successfully!");
            } else {
                System.out.println("Room already booked!");
            }
        } else {
            System.out.println("Invalid room selection!");
        }
        break;
    case 3:
        System.out.println("Exiting...");
        sc.close();
}

```

```

        return;

    default:

        System.out.println("Invalid choice!");
    }
}
}
}
}

```

Q6) Write a Java program to implement a Library Management System where books can be added, issued, and returned. The system should track the total number of books using a static field and allow users to view book details, issue or return books, and check the total book count using static methods.

```

import java.util.*;

class Book {
    int id;
    String name;
    boolean issued;
    static int totalBooks = 0; // static field

    Book(int id, String name) {
        this.id = id;
        this.name = name;
        this.issued = false;
        totalBooks++;
    }

    static void showTotalBooks() { // static method
        System.out.println("Total Books: " + totalBooks);
    }
}

public class LibrarySystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Book> list = new ArrayList<>();
    }
}

```

```

while (true) {
    System.out.println("\n1.Add Book 2.View Books 3.Issue 4.Return 5.Total 6.Exit");
    System.out.print("Enter choice: ");
    int ch = sc.nextInt();
    sc.nextLine(); // clear buffer
    switch (ch) {
        case 1:
            System.out.print("Enter Book ID: ");
            int id = sc.nextInt();
            sc.nextLine();
            System.out.print("Enter Book Name: ");
            String name = sc.nextLine();
            list.add(new Book(id, name));
            System.out.println("Book added!");
            break;
        case 2:
            for (Book b : list)
                System.out.println(b.id + " " + b.name + " " + (b.issued ? "Issued" : "Available"));
            break;
        case 3:
            System.out.print("Enter Book ID to issue: ");
            int iid = sc.nextInt();
            for (Book b : list) {
                if (b.id == iid && !b.issued) {
                    b.issued = true;
                    System.out.println("Book issued!");
                    break;
                }
            }
            break;
        case 4:
            System.out.print("Enter Book ID to return: ");

```

```

        int rid = sc.nextInt();
        for (Book b : list) {
            if (b.id == rid && b.issued) {
                b.issued = false;
                System.out.println("Book returned!");
                break;
            }
        }
        break;
    case 5:
        Book.showTotalBooks();
        break;

    case 6:
        System.out.println("Exiting...");
        sc.close();
        return;
    default:
        System.out.println("Invalid choice!");
    }
}
}
}
}

```

Q7) Create a Java program demonstrating single inheritance where a subclass extends a superclass and calls its methods.

```

class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {

```

```

        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // calling superclass method
        d.bark(); // calling subclass method
    }
}

```

Q8) Implement an interface in Java and create multiple classes that implement the interface, demonstrating polymorphism.

```

interface Shape {
    void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle implements Shape {
    public void draw() {
        System.out.println("Drawing Rectangle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s; // reference of interface
        s = new Circle(); // polymorphism
        s.draw();

        s = new Rectangle(); // polymorphism
        s.draw();
    }
}

```

```
}
```

Q9) Write a Java program to create an abstract class with an abstract method and extend it in a subclass that provides an implementation.

```
abstract class Shape {  
    abstract void area(); // abstract method  
}  
  
class Circle extends Shape {  
    double r = 5;  
    void area() {  
        System.out.println("Area of Circle: " + (Math.PI * r * r));  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Shape s = new Circle(); // reference of abstract class  
        s.area();           // calling implemented method  
    }  
}
```

Q10) Develop a Java application that simulates an ATM machine. Implement functionalities like checking account balance, withdrawing, and depositing money. Use try, catch, and finally blocks to handle potential exceptions such as insufficient funds (throwing `ArithmeticException`) and invalid input (throwing `IllegalArgumentException`). Ensure that the application continues to run smoothly after handling exceptions.

```
import java.util.*;  
  
class ATM {  
    double balance = 10000; // initial balance  
    void checkBalance() {  
        System.out.println("Balance: " + balance);  
    }  
  
    void deposit(double amt) {
```



```

        System.out.print("Enter amount: ");
        atm.withdraw(sc.nextDouble());
        break;
    case 4:
        System.out.println("Exiting...");
        sc.close();
        return;
    default:
        throw new IllegalArgumentException("Invalid choice!");
    }
} catch (InputMismatchException e) {
    System.out.println("Invalid input! Enter numbers only.");
    sc.nextLine(); // clear buffer
} catch (IllegalArgumentException | ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Transaction processed.");
}
}
}
}
}

```

Q11) Develop a Java application that simulates an online shopping system. Implement functionalities such as adding items to the cart, calculating the total price, and processing payments. Use try, catch, and finally blocks to handle exceptions like `NumberFormatException` for invalid input and `ArithmeticException` for any calculation errors.

```

import java.util.*;

class Item {
    String name;
    double price;
    int qty;

    Item(String name, double price, int qty) {

```

```

        this.name = name;

        this.price = price;

        this.qty = qty;
    }

    double total() {
        return price * qty;
    }
}

public class ShoppingSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Item> cart = new ArrayList<>();
        while (true) {
            System.out.println("\n1.Add Item 2.View Cart 3.Total 4.Pay 5.Exit");
            System.out.print("Enter choice: ");

            try {
                int ch = Integer.parseInt(sc.nextLine()); // for NumberFormatException
                switch (ch) {
                    case 1:
                        System.out.print("Enter name: ");
                        String name = sc.nextLine();
                        System.out.print("Enter price: ");
                        double price = Double.parseDouble(sc.nextLine());
                        System.out.print("Enter quantity: ");
                        int qty = Integer.parseInt(sc.nextLine());
                        cart.add(new Item(name, price, qty));
                        System.out.println("Item added!");
                        break;
                    case 2:
                        for (Item i : cart)
                            System.out.println(i.name + " " + i.price + " x " + i.qty);
                        break;
                }
            }
        }
    }
}

```

```

case 3:
    double total = 0;
    for (Item i : cart) total += i.total();
    System.out.println("Total: " + total);
    break;
case 4:
    double sum = 0;
    for (Item i : cart) sum += i.total();
    System.out.print("Enter payment amount: ");
    double pay = Double.parseDouble(sc.nextLine());
    if (pay < sum)
        throw new ArithmeticException("Insufficient payment!");
    System.out.println("Payment successful! Change: " + (pay - sum));
    break;
case 5:
    System.out.println("Exiting...");
    sc.close();
    return;
default:
    throw new IllegalArgumentException("Invalid choice!");
}
} catch (NumberFormatException e) {
    System.out.println("Invalid input! Enter numbers properly.");
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Operation completed.");
}
}
}

```

Q12) Develop a Java application that connects to a MySQL/PostgreSQL database to manage student information. Implement CRUD operations for student records using JDBC. Use prepared statements to handle SQL queries securely and ensure proper transaction management.

```
ps1.setInt(1, sc.nextInt());
```

```
ps1.setString(2, sc.next());
ps1.setInt(3, sc.nextInt());
ps1.executeUpdate();
System.out.println("Inserted!");
break;
```

case 2: // View

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM students");
while (rs.next())
    System.out.println(rs.getInt(1) + " " +
        rs.getString(2) + " " +
        rs.getInt(3));
break;
```

case 3: // Update

```
PreparedStatement ps2 = con.prepareStatement(
    "UPDATE students SET name=?, age=? WHERE id=?");
System.out.print("New Name Age ID: ");
ps2.setString(1, sc.next());
ps2.setInt(2, sc.nextInt());
ps2.setInt(3, sc.nextInt());
ps2.executeUpdate();
System.out.println("Updated!");
break;
```

case 4: // Delete

```
PreparedStatement ps3 = con.prepareStatement(
    "DELETE FROM students WHERE id=?");
System.out.print("Enter ID: ");
ps3.setInt(1, sc.nextInt());
ps3.executeUpdate();
System.out.println("Deleted!");
break;
```

```

        case 5:
            con.commit(); // commit all changes
            System.out.println("Exiting...");
            return;
        default:
            System.out.println("Invalid choice!");
    }
    con.commit(); // commit each successful operation
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
    con.rollback(); // rollback on error
}
}
} catch (Exception e) {
    System.out.println("Connection Error: " + e.getMessage());
}
}
}

```

Q13) Create a Java application that connects to a MySQL/PostgreSQL database to manage employee information. Implement functionalities like adding, updating, deleting, and viewing employee records using JDBC. Use prepared statements to prevent SQL injection and handle exceptions gracefully.

```

CREATE DATABASE test;

USE test;

CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    salary DOUBLE
);

```

```

import java.sql.*;
import java.util.*;

public class EmployeeJDBC {

    static final String URL = "jdbc:mysql://localhost:3306/test";
    static final String USER = "root";
    static final String PASS = "your_password"; // change this

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(URL, USER, PASS);

            while (true) {

                System.out.println("\n1.Add 2.View 3.Update 4.Delete 5.Exit");

                System.out.print("Enter choice: ");

                int ch = sc.nextInt();

                sc.nextLine();

                try {

                    switch (ch) {

                        case 1: // ADD

                            System.out.print("Enter ID: ");

                            int id = sc.nextInt();

                            sc.nextLine();

                            System.out.print("Enter Name: ");

                            String name = sc.nextLine();

                            System.out.print("Enter Salary: ");

                            double sal = sc.nextDouble();

                            try (PreparedStatement ps = con.prepareStatement(

                                "INSERT INTO employees VALUES (?, ?, ?)")) {

                                ps.setInt(1, id);

                                ps.setString(2, name);

                                ps.setDouble(3, sal);

                                ps.executeUpdate();

```

```

        System.out.println("Employee added!");
    }
    break;
case 2: // VIEW
    try (Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM employees")) {
        while (rs.next()) {
            System.out.println(
                rs.getInt("id") + " " +
                rs.getString("name") + " " +
                rs.getDouble("salary"));
        }
    }
    break;
case 3: // UPDATE
    System.out.print("Enter ID: ");
    int uid = sc.nextInt();
    sc.nextLine();

    System.out.print("New Name: ");
    String newName = sc.nextLine();
    System.out.print("New Salary: ");
    double newSal = sc.nextDouble();
    try (PreparedStatement ps = con.prepareStatement(
        "UPDATE employees SET name=?, salary=? WHERE id=?")) {
        ps.setString(1, newName);
        ps.setDouble(2, newSal);
        ps.setInt(3, uid);
        int rows = ps.executeUpdate();
        System.out.println(rows > 0 ? "Updated!" : "Record not found!");
    }
    break;

```

```

case 4: // DELETE

    System.out.print("Enter ID: ");

    int did = sc.nextInt();

    try (PreparedStatement ps = con.prepareStatement(
        "DELETE FROM employees WHERE id=?")) {

        ps.setInt(1, did);

        int rows = ps.executeUpdate();

        System.out.println(rows > 0 ? "Deleted!" : "Record not found!");

    }

    break;

case 5:

    System.out.println("Exiting...");

    sc.close();

    con.close();

    return;

default:

    System.out.println("Invalid choice!");

}

} catch (Exception e) {

    System.out.println("Error: " + e.getMessage());

}

}

} catch (Exception e) {

    System.out.println("Connection Error: " + e.getMessage());

}

}

}

```